

Digital Wallet & UPI Payment System

Object-Oriented Java Implementation | Custom Exception Handling & Validation

OBJECTIVE / AIM

To design and implement a **Digital Wallet System** in Java using **Object-Oriented Programming (OOP) principles**, simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, transaction validation, and custom exception handling.

PROBLEM STATEMENT & DESCRIPTION

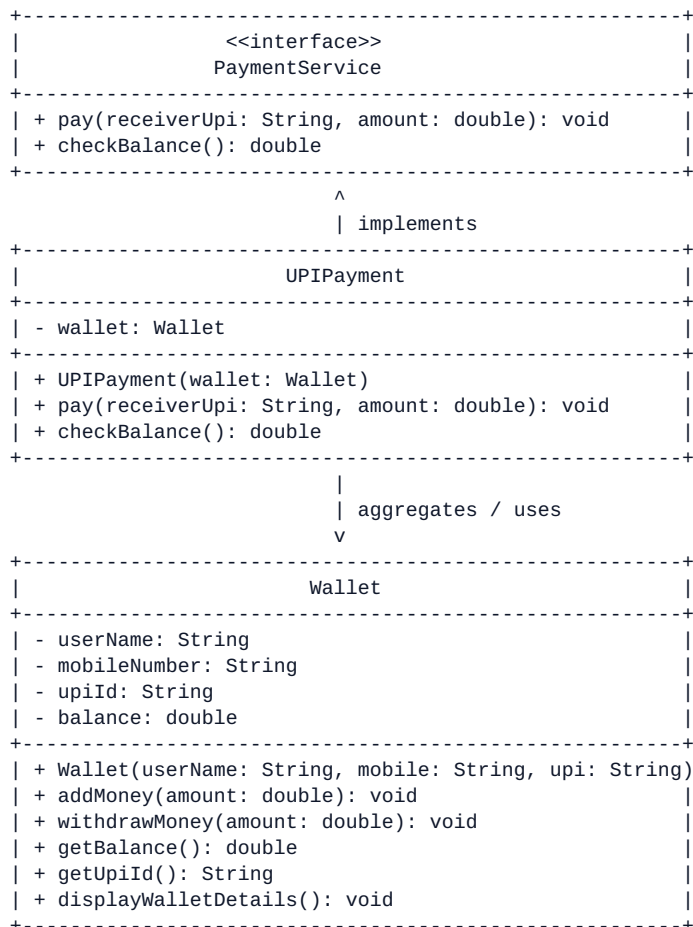
Modern payment applications (e.g., Paytm, Google Pay, PhonePe) require strict architectural controls to ensure data integrity, robust user authentication, and transaction safety. This project simulates a digital payment ecosystem featuring:

- **Encapsulation:** Sensitive data members (User Name, Mobile Number, UPI ID, Balance) are declared `private` within the `Wallet` class.
- **Abstraction & Polymorphism:** Defined via the `PaymentService` interface enforcing standardized behavior across implementation models.
- **Validation Checks:** Ensures UPI format contains @, non-negative/non-zero transfer amounts (`amount > 0`), and prevents overdrafts (`balance ≥ amount`).
- **Custom Exception Handling:** Graceful recovery and messaging for edge cases via custom exception hierarchies.

ALGORITHM

1. **Start** the system execution.
2. Define Custom Exception classes derived from `Exception`: `InsufficientBalanceException`, `InvalidUPIException`, and `InvalidAmountException`.
3. Define the `PaymentService` interface declaring signature methods `pay(String, double)` and `checkBalance()`.
4. Create the `Wallet` class featuring private fields: `userName`, `mobileNumber`, `upiId`, and `balance`.
5. Implement wallet state manipulators: constructor, `addMoney()`, `withdrawMoney()`, and `displayWalletDetails()`.
6. Implement the `UPIPayment` class derived from `PaymentService`, containing an aggregated reference to a `Wallet`.
7. In `UPIPayment.pay(receiverUpi, amount)`:
 - Validate `receiverUpi` string structure. If invalid, throw `InvalidUPIException`.
 - Validate transfer amount. If non-positive, throw `InvalidAmountException`.
 - Validate wallet funds. If balance is smaller than amount, throw `InsufficientBalanceException`.
 - Deduct funds upon successful validation and render receipt.
8. Execute test workflows in `Main` covering standard operations and negative test cases via `try-catch-finally` blocks.
9. Render final state and **End** program execution.

UML CLASS DIAGRAM



SOURCE CODE (JAVA)

```
/**
 * Title: Digital Wallet and UPI Payment System
 * Concepts: Interfaces, Encapsulation, Exception Handling
 */

// Custom Exception Classes
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidUPIException extends Exception {
    public InvalidUPIException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

// Payment Service Interface
interface PaymentService {
    void pay(String receiverUpi, double amount)
        throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException;
}
```

```

    double checkBalance();
}

// Encapsulated Wallet Class
class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    public Wallet(String userName, String mobileNumber, String upiId) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = 0.0;
    }

    public void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Deposit amount must be greater than zero.");
        }
        this.balance += amount;
        System.out.println(" SUCCESS: ₹" + amount + " added to wallet successfully.");
    }

    public void withdrawMoney(double amount) {
        this.balance -= amount;
    }

    public double getBalance() { return this.balance; }
    public String getUpiId() { return this.upiId; }

    public void displayWalletDetails() {
        System.out.println("\n-----");
        System.out.println("          CURRENT WALLET STATUS          ");
        System.out.println("-----");
        System.out.println(" Account Holder : " + userName);
        System.out.println(" Mobile Number  : " + mobileNumber);
        System.out.println(" UPI ID         : " + upiId);
        System.out.println(" Current Balance: ₹" + String.format("%.2f", balance));
        System.out.println("-----\n");
    }
}

// UPI Payment Implementation
class UPIPayment implements PaymentService {
    private Wallet wallet;

    public UPIPayment(Wallet wallet) {
        this.wallet = wallet;
    }

    @Override
    public void pay(String receiverUpi, double amount)
        throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException {

        System.out.println("\n--> Initiating Payment of ₹" + amount + " to [" + receiverUpi + "].");

        if (receiverUpi == null || !receiverUpi.contains("@") || receiverUpi.trim().length() < 5) {
            throw new InvalidUPIException("Transaction Failed: Invalid UPI ID format (" + receiverUpi + ").");
        }

        if (amount <= 0) {
            throw new InvalidAmountException("Transaction Failed: Transfer amount must be positive.");
        }

        if (wallet.getBalance() < amount) {
            throw new InsufficientBalanceException("Transaction Failed: Insufficient balance. Available: ₹"
                + wallet.getBalance() + ", Required: ₹" + amount);
        }
    }
}

```

```

    }

    wallet.withdrawMoney(amount);
    System.out.println(" SUCCESS: Payment of ₹" + amount + " sent to " + receiverUpi + "!");
}

@Override
public double checkBalance() {
    return wallet.getBalance();
}
}

// Driver Class
public class Main {
    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("          WELCOME TO DIGITAL UPI PAYMENT SYSTEM          ");
        System.out.println("=====");

        Wallet myWallet = new Wallet("Alex Morgan", "9876543210", "alex@upi");
        PaymentService paymentApp = new UPIPayment(myWallet);

        myWallet.displayWalletDetails();

        try {
            System.out.println("[ACTION]: Adding ₹5000 to Wallet...");
            myWallet.addMoney(5000.00);

            paymentApp.pay("merchant@okaxis", 1250.50);

            try {
                paymentApp.pay("invalid_upi_handle", 200.00);
            } catch (Exception e) {
                System.out.println(" EXCEPTION CAUGHT: " + e.getMessage());
            }

            try {
                paymentApp.pay("friend@okicici", -50.00);
            } catch (Exception e) {
                System.out.println(" EXCEPTION CAUGHT: " + e.getMessage());
            }

            try {
                paymentApp.pay("store@ybl", 10000.00);
            } catch (Exception e) {
                System.out.println(" EXCEPTION CAUGHT: " + e.getMessage());
            }

        } catch (InvalidAmountException e) {
            System.out.println(" ERROR: " + e.getMessage());
        } finally {
            System.out.println("\n[SYSTEM]: Closing session and rendering report...");
            myWallet.displayWalletDetails();
        }
    }
}

```

TERMINAL STYLE OUTPUT SCREENSHOT

```

=====
WELCOME TO DIGITAL UPI PAYMENT SYSTEM
=====

-----
CURRENT WALLET STATUS
-----
Account Holder : Alex Morgan

```

```
Mobile Number : 9876543210
UPI ID       : alex@upi
Current Balance: ₹0.00
-----

[ACTION]: Adding ₹5000 to Wallet...
SUCCESS: ₹5000.0 added to wallet successfully.

--> Initiating Payment of ₹1250.5 to [merchant@okaxis]...
SUCCESS: Payment of ₹1250.5 sent to merchant@okaxis!

--> Initiating Payment of ₹200.0 to [invalid_upi_handle]...
EXCEPTION CAUGHT: Transaction Failed: Invalid UPI ID format (invalid_upi_handle).

--> Initiating Payment of ₹-50.0 to [friend@okicici]...
EXCEPTION CAUGHT: Transaction Failed: Transfer amount must be positive.

--> Initiating Payment of ₹10000.0 to [store@ybl]...
EXCEPTION CAUGHT: Transaction Failed: Insufficient balance. Available: ₹3749.5, Required: ₹10000.0

[SYSTEM]: Closing session and rendering report...

-----
CURRENT WALLET STATUS
-----
Account Holder : Alex Morgan
Mobile Number  : 9876543210
UPI ID        : alex@upi
Current Balance: ₹3749.50
-----
```

RESULTS & CONCLUSION

Conclusion:

requirements for the Digital Wallet & UPI Payment System were successfully designed, implemented, and verified in Java. The application correctly encapsulates private user details, validates incoming transaction data against logical constraints, and safely processes exceptions without runtime crashes.